Mandate, full documentation

Team 10, Raingentic Commerce Hackathon NYC, 2026-08-08/09.

---

1. What it is

An agent gets a **scoped virtual Rain card** bound to the exact purchase order it negotiated: vendor, SKU, price, quantity, expiry. Deterministic code checks the declared order against the real record before the card is ever created. Any mismatch means no card, not a decline, an instrument that never comes into existence.

Rain's own products bound *how much* an agent spends and *where*. Mandate binds *why*, at the same moment, one step earlier.

---

2. The architecture

```
1 TASK        an agent is given a job
     |
1b NEGOTIATE  competing sellers bid, one counter-offer round, a winner
     |          the winning quote is written to the record as an accepted order line
     v
2 PROPOSE     the agent declares that quote as a purchase order
     |          { poNumber, vendor, sku, unitPrice, quantity, quoteExpiry, costCentre }
     v
3 VERIFY      eleven deterministic checks against a record snapshot   <-- no model here
     |          rules are versioned CONFIG, not code
     |
     |-- fails ------> REFUSE  no card, ever. A plain-English reason. Logged.
     |
     |-- too big ----> HOLD    every check passed, but it is above the delegated
     |                         limit. Still no card. A named person releases it,
     |                         and every check runs again on fresh records.
     v
4 ISSUE       Rain issues a virtual card scoped to exactly this PO
     v
5 SETTLE      the cost centre is charged, the record is written back
     v
6 REVOKE      the card is retired — it existed for exactly this obligation
     v
7 RECORD      PO + snapshot + rule version + every check + outcome, append-only
```

There is no code path where a card is created and then judged, and none where a card outlives the obligation that justified it. The refusal and hold branches never reach Rain at all — that is enforced by the shape of `lib/pipeline.ts`, not by a flag.

The eleven checks that decide stage 3, in order: the PO exists and was accepted; it is still open and the quote has not expired; the amount matches the quote; the vendor **and SKU** match; it is within the cost centre's budget; no card has already been issued for this line; it is inside the company's delegated limit; it is

not a large purchase split in two to duck that limit; it is inside this particular agent's own limit; the vendor is one we have paid before; and the agent is not spending faster than it should.

The last four read **patterns rather than moves**. Splitting a purchase to avoid an approval is not a lie — every individual line is true — so only a rule looking at the running total can see it.

Monad: each **rule version** is hashed and anchored on testnet. This is what proves the rules were not edited after the fact to fit a history someone already had, closing the one real hole in an otherwise-airtight audit story.

2b. The six pages, and what each one is for

The site grew a page at a time, so this is the map. It is the same list the navigation is built from, in `components/SiteNav.tsx`.

	Page	Route	What it is for
1	**Workspace**	`/workspace`	What a customer sees — plain language, approvals, budgets
2	**Console**	`/`	Run it and watch the checks. Every panel, every control
3	**Catalogue**	`/catalog`	What an agent can buy
4	**Agents**	`/agents`	Who is allowed to spend, and how much each may spend alone
5	**System design**	`/architecture`	How it works, and this same page map
6	**Deck**	`/presentation`	The pitch

Two of these are the product; the rest explain it. Workspace is what someone who bought this would use day to day. Console is the same system with its working shown — every check, every record it read, and the controls to change policy and re-judge history. Same data, same API, same decisions; only the amount of machinery on screen differs.

One navigation bar appears on all six, in that order, plus a second "on this page" row inside the workspace for its own sections. Earlier each page carried its own bespoke header, so where you could go next depended on where you happened to be — the console had no way out at all, and the workspace had a second nav that led nowhere else in the site.

---

3. Why this company, specifically

Rain is not a generic payments API. It is a **Visa and Mastercard Principal Member**, meaning it issues cards directly rather than reselling someone else's licence, live across 175 million merchant locations in 220+ countries, backing 100+ programs on $250M raised. Its newest product, shipped this June, is the **Agent Control Layer**: programmatic guardrails, amount limits, merchant allowlists, spend intervals, enforced "*at card issuance and transfer initiation rather than applied after the fact.*"

That last phrase is Mandate's whole thesis. Rain already committed, publicly, to enforcement-before-the-fact as the right design. Mandate is the same principle carried one layer higher, from "can this agent spend this much, here" to "is this specific declared reason for spending actually true." It does not compete with the Agent Control Layer. It sits on it.

Concretely, the build uses:

- **Rain's collateral-backed card issuance** (`issueScopedCard` in `lib/rain-client.ts`), the same primitive shown live in Rain's own "Wednesday We Wear Pink" reference demo during the workshop: onramp → store stablecoins → user authorizes agent → **scoped virtual card issued, locked to merchant and amount** → agent completes the transaction → **card retired automatically once the job is done.** Mandate's issue/settle/revoke stages are that exact flow.
- **The team's own collateral contract**, pre-provisioned (`RAIN_COLLATERAL_CONTRACT_ID`), meaning the KYC and contract-deployment steps Rain normally requires were already done for Team 10 by Rain, and the build starts at the step that matters, issuance.

---

4. Why this hackathon, specifically

Three of the five judges work at Rain: Ross Basri (product lead, launched Rain Rewards), Farhan Khwaja (engineer, high-throughput transactional systems, custody infrastructure), Juan Blanco (data engineer, has personally won Ethereum hackathons in Amsterdam and Paris, so he will recognize a staged demo on sight). The other two are Siggy Bilstein (Engineering Manager at Cursor, works on an agent-native git forge) and Jarrod Watts (AI Engineering Lead at Monad, works specifically on agent orchestration).

This shaped four concrete build decisions:

- **The lead demo moment is unfalsifiable, not scripted.** Issue a card for a PO, then submit the identical PO again. It is refused by the idempotency check, reading the record the first run itself wrote. Nothing is authored for the demo. This exists because Juan Blanco would spot a hand-authored "bad agent" fixture in seconds.
- **No LLM anywhere in the verify path.** Every check in `lib/verify.ts` (B's side) is a pure function. This is what makes replay meaningful, editing a rule and re-running history only proves something if the thing being re-run is deterministic. Farhan builds transactional systems for a living; "trust me, the agent judged it fairly" does not survive contact with that judge.
- **The rule-version Monad anchor, not a per-decision one.** Anchoring every decision is a gesture. Anchoring each rule version is structural, it is what proves the rules were not quietly rewritten to fit a history after the fact, and it is the honest answer to Jarrod's own stated bounty bar: "*the chain has to matter, would it break at 15 second finality or 50 cents a transaction?*" You can only afford to anchor rule history because Monad is this cheap.
- **The negotiation stage stays deliberately thin**, one vendor comparison rather than a multi-agent haggling performance, specifically because Jarrod's actual job is agent orchestration and a shallow simulated negotiation is the easiest thing in the room for him to see through.

---

5. How it covers all four submission categories

The event's tracks are not mutually exclusive submissions, one build can qualify for several with different framing. Rain's own workshop slide named four shapes for "Autonomous spend": **Autonomous spend, Global money movement, Treasury and payouts, Agent negotiation.** Mandate maps to two of Rain's own four, plus the general track and the Monad bounty:

Category	How Mandate qualifies
Best use of Rain	Card issuance is the enforcement point, matching Rain's own architecture and their published "at issuance, not after the fact" principle exactly
General track, "agents actually move money"	An agent runs a task end to end, budgets are debited, every decision is filed; settlement across card rails once issuance is live
Agent negotiation	Four sellers with distinct strategies and a counter-offer round produce the accepted PO the card is bound to — negotiation *causes* what gets verified, rather than running beside it
Monad bounty	Each rule version is hashed for anchoring on testnet, where the chain's low cost is structurally why the audit claim can hold at all. The write itself is the remaining gap

One pipeline, one architecture, two owners. Not four separate projects chasing four prizes.

---

6. What was deliberately not built, and why

Documented in full in `IDEAS.md`, `THE-IDEA.md`, and `ABI-LESSON.md`. In short:

- **Delegated budget trees** — too close to Rain's own program-level caps, would read as rebuilding their product rather than extending it.
- **Agent Underwriting** (a second collateral pool taking on liability) — needs Rain identities the team was not issued. Kept as one forward-looking sentence in the close.
- **Burn Rate streaming limits** — needs an unconfirmed Rain capability (live limit updates post-issuance) plus a Monad streaming contract from scratch. Two coupled unknowns on one afternoon.
- **Four-Eyes consensus voting** — rejected on principle, not just time. LLM agents voting reintroduces a model into the verify path, which directly undoes the reason replay works at all.
- **A per-decision Monad fee, a live budget meter, a judge-editable form** — all real, all kept as optional additions, none load-bearing, all cut before anything in the core loop.

This scoping follows directly from a prior hackathon loss (Pulse Foundry × ABI Frameworks, 2026-06-28): the team built roughly 85% of the winning system's substance and still lost, to a team whose actual edge was a versioned, biller-editable rules config and a "re-run instantly" toggle. Mandate's replay feature is that same idea, built properly, from the start, rather than bolted on after losing to it once.

---

7. Stack and repo layout

Next.js 14 App Router, TypeScript, Tailwind. Deployed on Vercel, required, locally-hosted submissions are disqualified per the event's own rules.

```
lib/
  types.ts           shared PurchaseOrder / VendorQuote / ScopedCard shapes (A + B contract)
  money.ts           integer cents, never floats
  rain-client.ts     Rain API wrapper (A)
  negotiation.ts     competing sellers, strategies, one counter-offer round (A)
  sellers.ts         seller fixtures per task (A)
  agent.ts           task -> negotiation -> PurchaseOrder (A)
  llm.ts             optional seller dialogue, upstream of PROPOSE, never in verify (A)
  checks/            the eleven checks + verify(), pure functions, 82 tests (B)
  rules/             versioned rule data + the sha256 anchored on Monad (B)
  replay/            re-judging stored decisions against another rule version (B)
  store/             append-only log; Postgres when DATABASE_URL is set, memory otherwise (B)
  pipeline.ts        every stage in order — this file IS the architecture diagram (B)
  monad/anchor.ts    publishing a rule version's hash to Monad testnet (B)
  rain/issuer.ts     the issue seam: verify passes -> rain-client is called
  seed/              54 committed historical decisions, deterministically generated (B)
app/
  page.tsx           the console        /
  workspace/         the customer view  /workspace
  catalog/           what can be bought /catalog
  agents/            who may spend      /agents
  architecture/      how it works       /architecture
  presentation/      the pitch          /presentation
  api/state/         everything the console and workspace render
  api/run/           run a task, or a hand-written PO, through the same pipeline
  api/purchase/      the full journey: negotiate -> propose -> verify -> issue
  api/approve/       release a held purchase; a named person, checks re-run
  api/rules/         list rule versions, propose an edit as the next version
  api/rules/activate/ the second pair of eyes — the author may not approve their own
  api/replay/        re-judge history against an edited rule version
  api/anchor/        publish a rule version's hash to Monad
  api/reset/         restore the seeded state between demos
  api/rain/ping/     the first-authenticated-call milestone
components/
  SiteNav.tsx        PAGES — the single source of truth for navigation and the page map
docs/                this file, SUBMISSION.md, CHANGELOG.md, and the full planning trail
```

Status, stated plainly so nothing is overclaimed in front of judges. Built and tested: the eleven checks, rule versioning with dual control, replay, the append-only log, the negotiation, human escalation and release, card retirement, idempotency under concurrency, and the run-it-twice refusal.

Card issuance returns a **simulated** card, labelled as such in the UI. The auth header and the issuance path are both **confirmed against Rain's live sandbox**, and our KYC is approved — but no collateral contract is linked to the user, so a card issued today would have no spending power. That is the one thing standing between this and money genuinely moving, and it is a question for a Rain engineer rather than a piece of code.

The Monad anchor is **written and wired** but inert: it needs a testnet RPC URL and a funded key, and the anchor control simply does not appear without them. Nothing in the decision path depends on the chain being reachable.

See `SUBMISSION.md` §5 for the honest per-track breakdown, `RAIN-API-CONFIRMED.md` for what the sandbox actually told us, and `CHANGELOG.md` for how each piece arrived.

Repo: `github.com/theomthakur/raingentic-team10`, public per the event's rules, `.env.local` never committed, only `.env.local.example`.